A Comparative Analysis of Optimization Trajectories and Convergence Properties.

Ozair Faizan
ahmad.faizan@epfl.ch

Paul Tercier
paul.tercier@epfl.ch

***Abstract*—The choice of the optimization algorithm determines training efficiency and generalization of deep neural networks. While methods such as Stochastic Gradient Descent and Adam have become field standards, recent works such as Lion [Che+25] and Muon [Jor+24] promise better convergence properties in practice. In this paper we present a comparative study of the optimization trajectories produced by Stochastic Gradient Descent, Adam, Lion and Muon applied to Multi-Layer Perceptron on the Fashion-MNIST dataset [XRV17]. To this end, we use a linear interpolation method: for parameters $\boldsymbol{\theta}_t$ before and $\boldsymbol{\theta}_{t+1}$ after an epoch, we evaluate the loss function at $N = 50$ points along the connecting vector, allowing us to empirically verify convexity and smoothness properties along the trajectory.**

Furthermore, we evaluate the convergence points $\boldsymbol{\theta}$ by testing the condition:

$$f(\boldsymbol{\theta}) \leqslant f(\boldsymbol{x}) + \nabla f(\boldsymbol{\theta})^{\mathsf{T}}(\boldsymbol{\theta} - \boldsymbol{x}),$$

for points $\boldsymbol{x}$ within a neighborhood $B_\varepsilon(\boldsymbol{\theta})$.

We use constant learning rate, and batch-size found by grid-search, all other hyperparameters are set to default.

I. Introduction

The choice of the optimizer is central to the process of training deep neural networks, influencing not only the speed of convergence but also the ability of the model to generalize. There is a vast number of available optimizers that can be applied to train a deep neural network, [SDH21] identified over one hundred unique algorithms, yet only two have dominated the scene: Stochastic Gradient Descent (SGD) [RM51] and Adam [KB15]. The Lion optimizer, which was developed based on symbolic search for update equations, is able to achieve similar levels of effectiveness without having problems with large amounts of memory usage. Furthermore, the Muon optimizer, developed initially to speed up the NanoGPT training, has shown better convergence rates.

Empirical comparisons typically report aggregate metrics such as final test accuracy or training loss curves. Less studied is the geometry of the optimization trajectory itself: whether successive parameter updates traverse locally convex regions of the loss landscape, how smooth the loss appears along each step, and whether convergence points satisfy local optimality conditions. Our analysis shows behavioral differences between optimizers that aggregate metrics alone may not reveal.

II. Models and Methods

We fit an MLP on the Fashion MNIST dataset which includes 10 different classes for image classification. This dataset contains a total of 60,000 gray-scale images of size 28×28. Our model is composed of two hidden layers of width 256 and 128 respectively, each with ReLU activation, followed by a linear classification layer, giving a total of approximately 234K trainable parameters. The cross-entropy loss is used with no regularization nor weight-decay for simplicity.

The following four optimization algorithms are compared. Stochastic gradient descent updates its parameters using

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \boldsymbol{g}_t$$

Here, $\boldsymbol{g}_t$ represents the stochastic gradient. The Adam optimizer computes the first- and second-moment averages of the gradient and performs bias-corrected adaptive steps per coordinate. The Lion optimizer uses only the sign of a momentum-gradient sum as the update direction:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \operatorname{sign}\big(\beta_1 \boldsymbol{m}_t + (1-\beta_1)\boldsymbol{g}_t\big),$$

where $\boldsymbol{m}_t$ is the momentum, updated as

$$\boldsymbol{m}_{t+1} = \beta_2 \boldsymbol{m}_t + (1-\beta_2)\boldsymbol{g}_t$$

The Muon optimizer implements the Nesterov-type momentum in the direction of steepest descent in terms of the spectral norm and approximates it by performing the Newton-Schulz orthogonalization of the gradient matrix.

The hyperparameters used for our analysis are found by performing a grid search over learning rates in $\{10^{-4}, 10^{-3}, 10^{-2}\}$ and batch sizes in $\{32, 64, 128\}$ and selecting the combination that achieves the lowest training loss for each optimizer after epoch 20. All other hyperparameters are kept at their published defaults.

For each optimizer, the parameter vectors $\boldsymbol{\theta}_t$ and $\boldsymbol{\theta}_{t+1}$ are stored at each epoch for a total of 5 epochs. The loss over the training set is computed at the points along the line

$$\boldsymbol{\theta}(\alpha) = (1-\alpha)\boldsymbol{\theta}_t + \alpha \boldsymbol{\theta}_{t+1},$$

for $\alpha \in \{0, 1/N, \ldots, 1\}$ and $N = 50$.

This allows us to analyze local approximation of the loss along the "average" trajectory.

We use the trajectory to verify convexity by plotting the values, and we compute the empirical smoothness factor along the trajectory,

$$L = \max_i \frac{\|\nabla f(\boldsymbol{\theta}_i) - \nabla f(\boldsymbol{\theta}_{i+1})\|}{\|\boldsymbol{\theta}_i - \boldsymbol{\theta}_{i+1}\|},$$

where $i \in \{0, \ldots, N-1\}$ indexes the interpolated points and the gradients are computed via finite differences.

Furthermore, we verify the first-order characterization of convexity by sampling $K = 50$ points on the ball $B_\varepsilon(\boldsymbol{\theta})$ for $\varepsilon \in \{10^{-2}, 1, \|\nabla f(\boldsymbol{\theta})\|, 10\|\nabla f(\boldsymbol{\theta})\|\}$, and checking,

$$f(\boldsymbol{\theta}) \leqslant f(\boldsymbol{x}) + \nabla f(\boldsymbol{\theta})^\mathsf{T}(\boldsymbol{\theta} - \boldsymbol{x}),$$

providing an empirical measure of local optimality.

III. Results

The selected learning rates and batch-sizes along with final train loss and test accuracy after 20 epochs are shown in Table I.

TABLE I
SELECTED LEARNING RATES AND FINAL PERFORMANCE AFTER 20 EPOCHS.

Optimizer	LR	Batch Size	Train Loss	Test Acc. (%)
SGD	10^{-2}	32	0.2383	88.38
Adam	10^{-3}	64	0.1426	88.66
Lion	10^{-4}	128	0.1269	88.52
Muon	10^{-3}	64	0.0600	90.01

Fig. 1 shows the interpolation of the losses for each of the 5 epochs, the empirical smoothness factors are summarized in Table II.

TABLE II
EMPIRICAL SMOOTHNESS FACTORS ALONG THE TRAJECTORY.

Epoch	SGD	Adam	Lion	Muon
1	9.0623	9.1043	20.157	7.3758
2	0.0936	0.2062	0.1640	0.1395
3	0.0872	0.4092	0.9197	0.0470
4	0.0926	0.4451	0.1061	0.0282
5	0.0679	0.2878	0.1345	0.0307

Finally, the local optimality condition,

$$f(\boldsymbol{\theta}) \leqslant f(\boldsymbol{x}) + \nabla f(\boldsymbol{\theta})^\mathsf{T}(\boldsymbol{\theta} - \boldsymbol{x}),$$

is satisfied by all $K = 50$ points across all ε and optimizers!

IV. Discussion

The comparative analysis of SGD, Adam, Lion, and Muon on the Fashion-MNIST dataset reveals distinct geometric behaviors in their optimization trajectories. While all four optimizers successfully reduce the training loss over time, the "path" they take through the parameter space varies significantly in terms of smoothness and local curvature.

Our linear interpolation analysis Fig. 1 highlights a transition in the loss landscape. During the first epoch, the trajectories traverse regions of non-convexity, suggesting that in early stages of training, the optimizers are traversing more complex landscapes due to the random initialization of parameters.

From the second epoch onwards, all four optimizers enter regions where the loss appears locally convex along the update vector, suggesting the optimizers settle into a

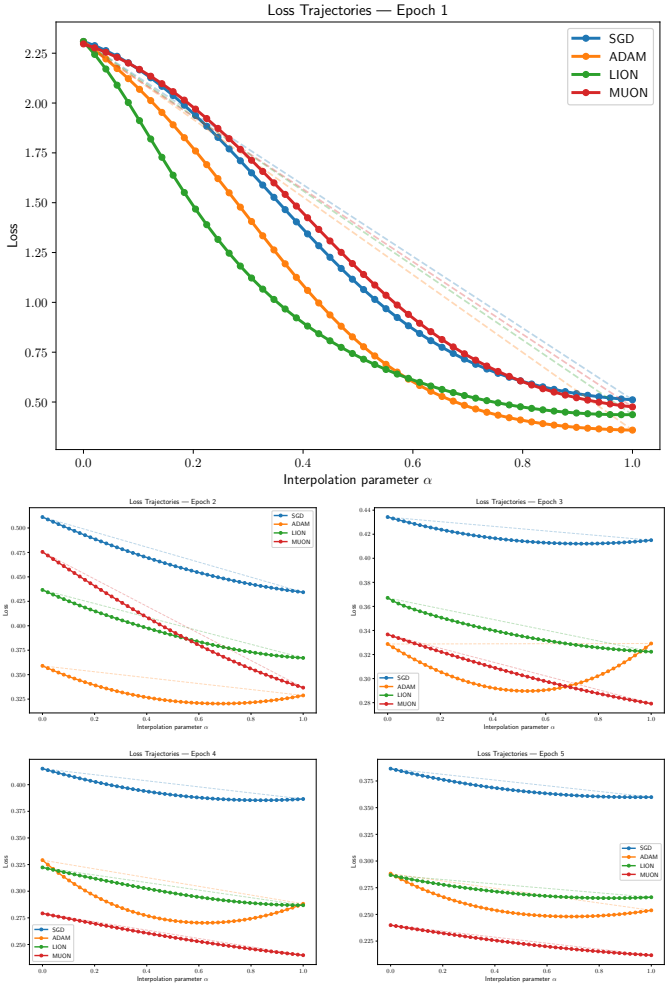

Fig. 1. Loss evaluated along the linear interpolation $\boldsymbol{\theta}(\boldsymbol{\alpha}) = (1-\alpha)\boldsymbol{\theta}_t + \alpha\boldsymbol{\theta}_{t+1}$ for each epoch. Dashed lines show the linear interpolation of endpoint losses.

regime where local geometry is better behaved. However we see that all optimizers except Muon step over points where the loss is lower than the endpoint. Muon on the other hand follows a linear decrease along the followed direction. Muon's trajectory becoming nearly linear suggests that it finds "highways" in the loss landscape.

The empirical smoothness factors Table II provide quantitative insight into these trajectories. All optimizers exhibit a massive spike in L during the first epoch, indicating high gradients and rapid changes in the landscape. Notably Lion shows the highest initial smoothness factor, which may be attributed to its use of the sign operator, forcing steps that can lead to sharper transitions.

The results of the local optimality test are particularly surprising. The fact that the first-order characterization of convexity held for all sampled points across every optimizer suggests that the final parameters $\boldsymbol{\theta}$ reside in a flat and well-behaved basins. Even for larger perturbations ε scaled by the gradient magnitude, the loss surface

did not exhibit the sharp non-convexities often theorized in high-dimensional landscapes. This empirical verification indicates that while the global landscape is non-convex, the optimizers are effective at finding convex regions.

Despite initially being highly non-convex, Muon outperforms the other optimizers by substantial margin, with better trajectory properties. This supports the hypothesis that the spectral norm-based updated of Muon provide a superior iductive bias for training deep architectures compared to the Adam or the sign-based updates of Lion.

V. Summary

This study compared the trajectories of SGD, Adam, Lion, and Muon on a standard MLP architecture. Our findings demonstrate that while all optimizers eventually reach locally convex regions, their paths to convergence differ geometrically. Muon emerged as the most efficient, not only achieving the lowest training loss (0.0600) and highest test accuracy (90.01%), but also maintaining the most linear and smooth trajectory throughout training.

Lion, while efficient in memory usage, exhibited higher initial smoothness factors, likely due to the discontinuous nature of the sign function in its update rule. Adam and SGD remained reliable benchmarks but were surpassed by Muon's ability to navigate the landscape without overshooting local minima along its update vector. Ultimately, our analysis suggests that newer algorithms like Muon offer superior inductive biases for neural network optimization, potentially reducing the number of iterations required to reach high-quality local minima in more complex, large-scale applications.

References

[Che+25] Lizhang Chen et al. *Lion Secretly Solves Constrained Optimization: As Lyapunov Predicts*. 2025. arXiv: 2310.05898 [cs.LG]. URL: https://arxiv.org/abs/2310.05898.

[Jor+24] Keller Jordan et al. *Muon: An optimizer for hidden layers in neural networks*. 2024. URL: https://kellerjordan.github.io/posts/muon/.

[KB15] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2015. arXiv: 1412.6980 [cs.LG]. URL: https://arxiv.org/abs/1412.6980.

[RM51] Herbert Robbins and Sutton Monro. "A Stochastic Approximation Method". In: *The Annals of Mathematical Statistics* 22.3 (1951), pp. 400–407. URL: https://doi.org/10.1214/aoms/1177729586.

[SDH21] Frank Schneider, Felix Dangel, and Philipp Hennig. "Descending through a Crowded Valley — Benchmarking Deep Learning Optimizers". In: *Proceedings of the 38th International Conference on Machine Learning (ICML)*. 2021. arXiv: 2007.01547 [cs.LG]. URL: https://arxiv.org/abs/2007.01547.

[XRV17] Han Xiao, Kashif Rasul, and Roland Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. Aug. 28, 2017. arXiv: cs.LG/1708.07747 [cs.LG].